TP7 - Data export and consolidation with SSIS

Exercise 1: Exporting sales to five destinations

Objective and source data

This exercise uses `dbo.Ventes2024` from TP6, stored in `TP6_Ventes` on the default SQL Server instance. The objective is to read the table once and export the same rows to CSV, Excel, Access, Oracle XE and PostgreSQL through a single SSIS Data Flow Task.

The source contains three sales:

Id	Annee	Region	Montant
1	2024	Tanger	1500.00
2	2024	Rabat	2300.00
30	2024	Tanger	500.00

The row with Id 30 was committed during the distributed transaction test in TP6. It remains part of the source, so each destination must receive **three rows**.

Package structure

The package `Sales_Export.dtsx` was created in Visual Studio with the SSIS extension. It contains one Data Flow Task named **Sales export**.

An **OLE DB Source** reads `Id`, `Annee`, `Region` and `Montant` from `dbo.Ventes2024`. It connects to SQL Server using the Microsoft OLE DB Driver for SQL Server and Windows Authentication.

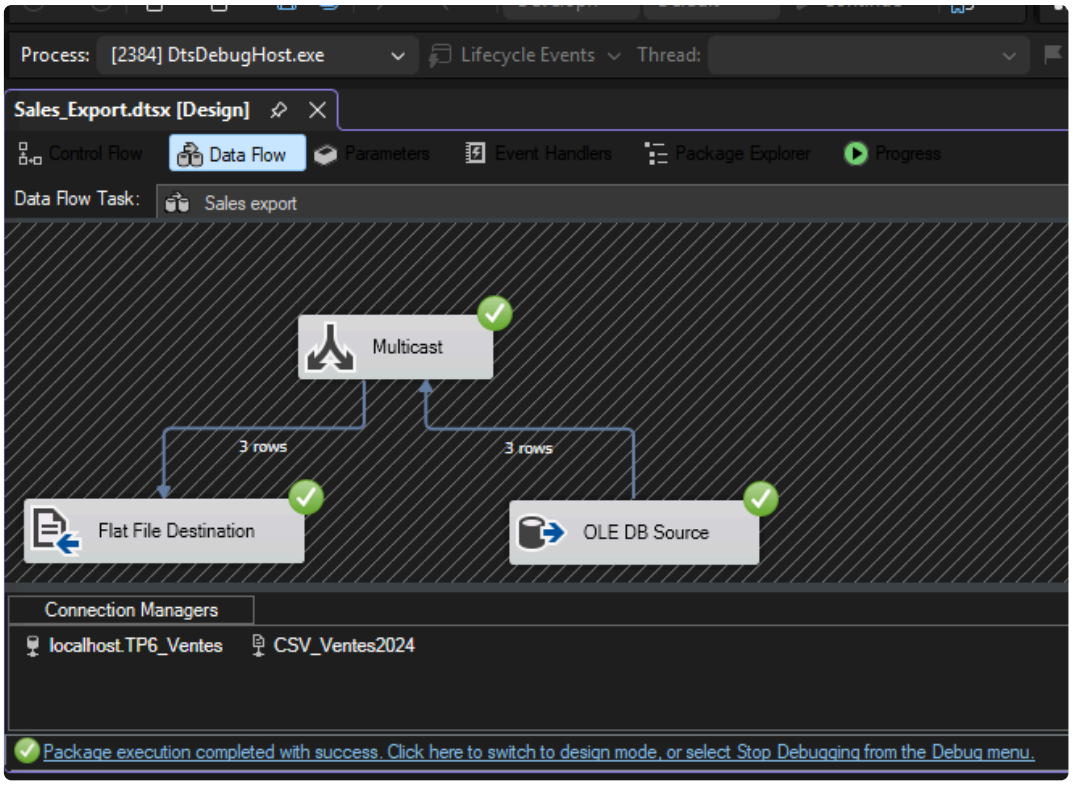
The source output feeds a **Multicast** transformation, which sends every input row to each of its five outputs. All destinations therefore receive the complete dataset from one source read.

CSV export

The first output writes to `C:\TP7\ventes2024.csv` through a **Flat File Destination**. The file uses semicolon separators, Windows line endings and Unicode text. The destination overwrites the existing file and writes the following header explicitly:

```
Id;Annee;Region;Montant
```

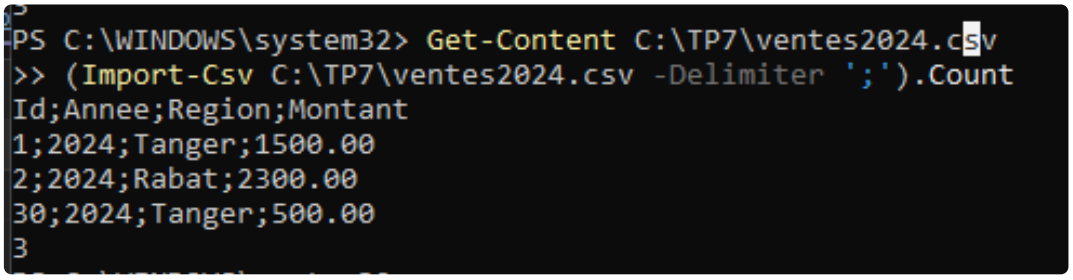
All four input columns are mapped to their corresponding file columns. The initial execution confirmed that three rows passed through the source and Multicast to the file destination.



The final file was checked in PowerShell:

```
Get-Content C:\TP7\ventes2024.csv
(Import-Csv C:\TP7\ventes2024.csv -Delimiter ';').Count
```

The output contains the header followed by the three sales. `Import-Csv` returns **3**, excluding the header.



Excel, Access, Oracle and PostgreSQL destinations

Four additional outputs were connected to the same Multicast transformation.

Destination	Target	Connection
Excel	`C:\TP7\ventes2024.xlsx`	Excel connection manager using ACE
Access	`SalesExport` in `C:\TP7\ventes.accdb`	ACE OLE DB
Oracle XE	`TP6.SALES_EXPORT` in `XEPDB1`	Oracle Provider for OLE DB
PostgreSQL	`public.sales_export` in `tp6_ventes`	PostgreSQL ODBC connection

The Excel destination writes the four fields to a worksheet with column names in the first row. The Access destination uses a `SalesExport` table prepared with compatible integer, text and decimal fields.

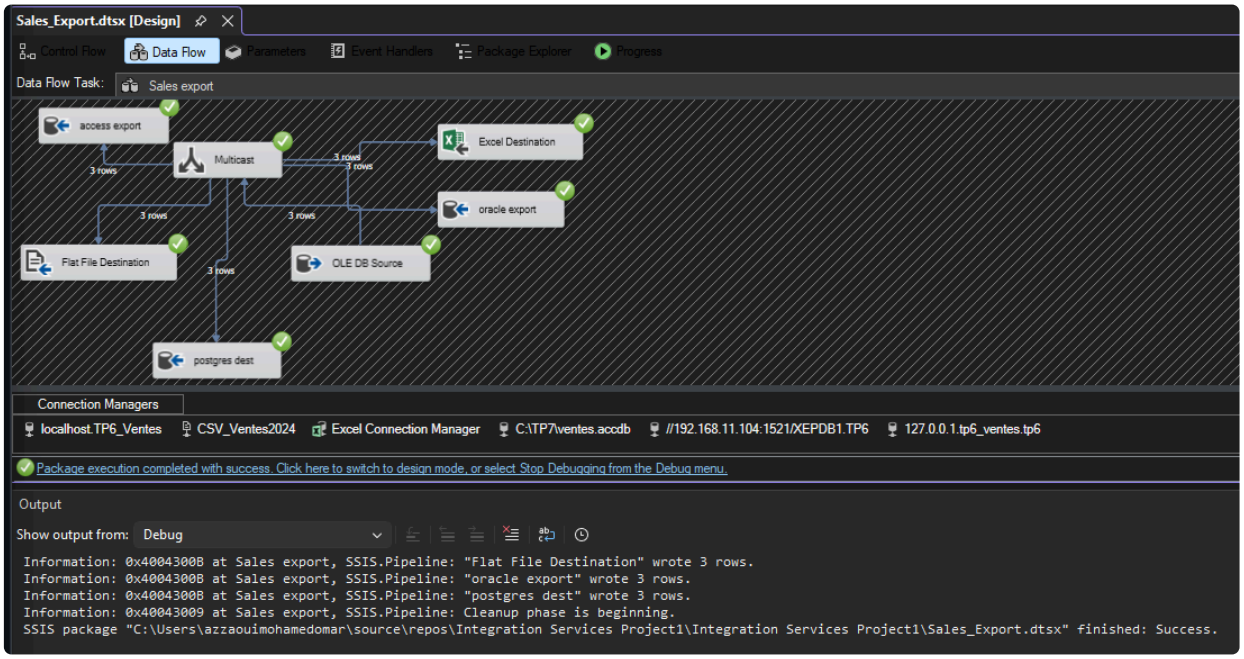
Oracle and PostgreSQL use English destination column names. The mappings preserve the meaning of the existing TP6 fields:

Source field	Oracle field	PostgreSQL field
Id	ID	id
Annee	SALE_YEAR	sale_year
Region	REGION	region
Montant	AMOUNT	amount

The Oracle connection uses the `TP6` account in the `XEPDB1` pluggable database. The PostgreSQL connection uses the `tp6` account and the `tp6_ventes` database prepared in TP6.

Complete package execution

The complete package executed successfully. The source, Multicast and all five destinations show successful execution, with **three rows sent to each destination**. The execution output confirms that the package finished with **Success**.



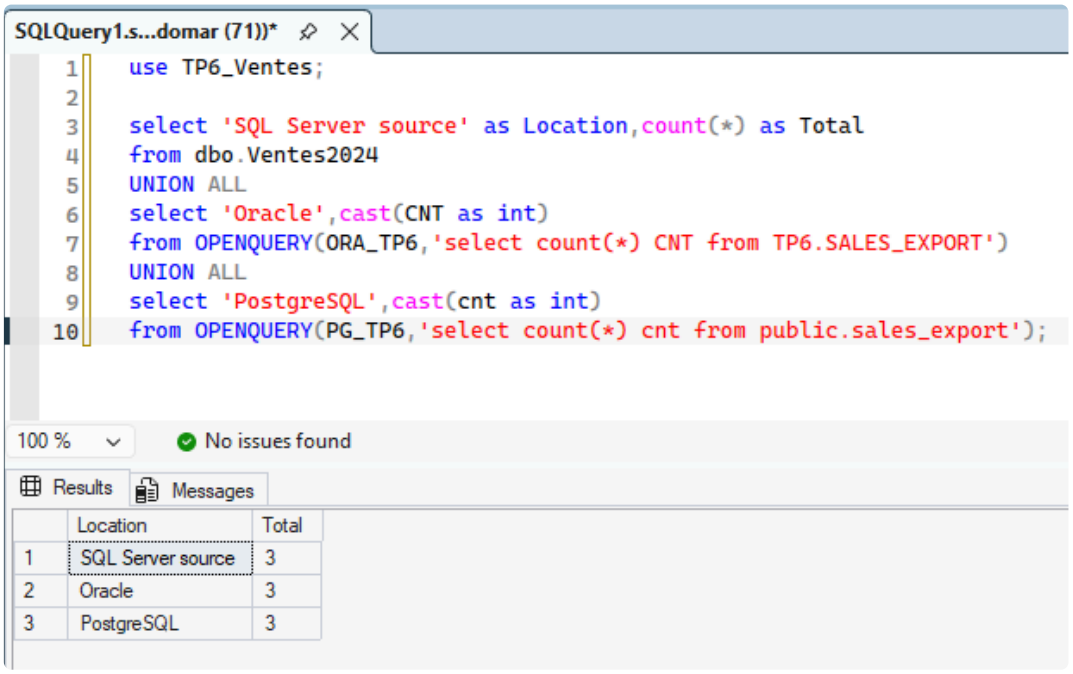
Verification of the exported data

A verification query was executed in SSMS on the default instance. It counts the local source rows and queries the Oracle and PostgreSQL destination tables through the existing linked servers:

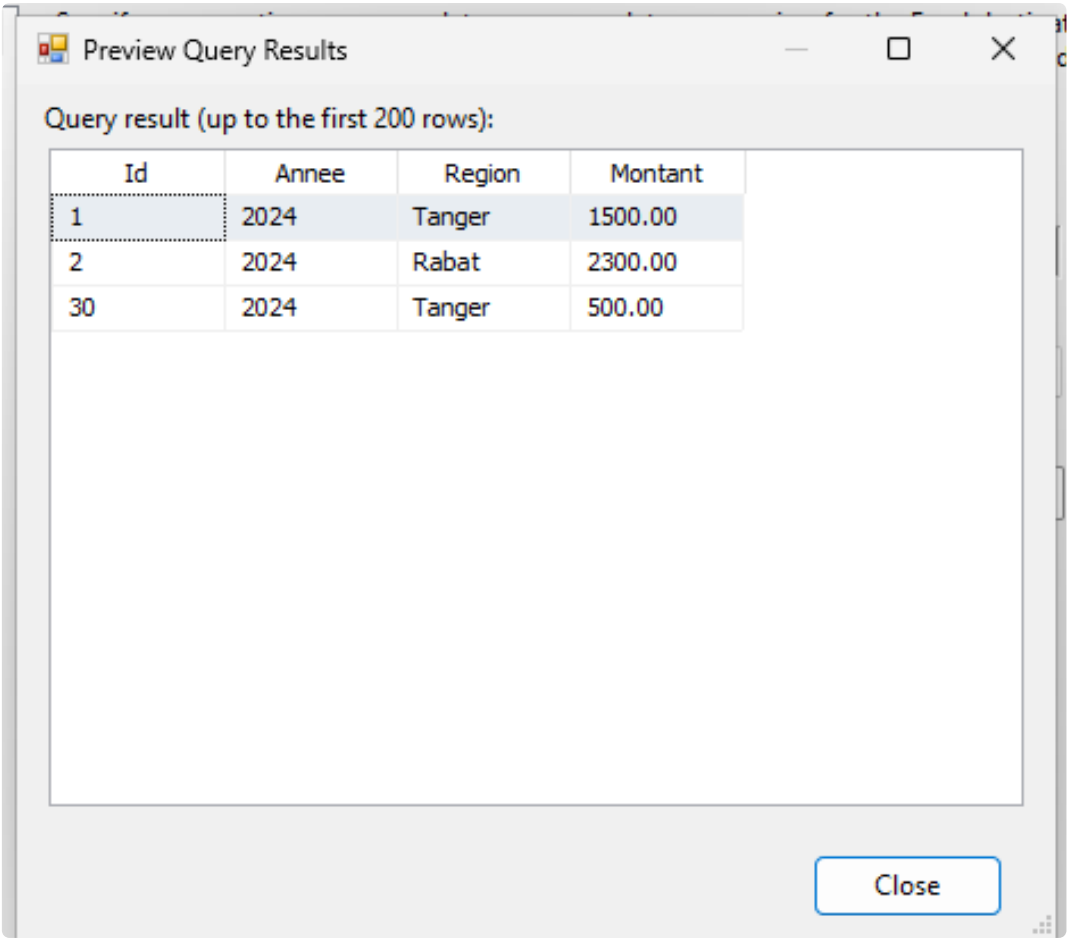
```
use TP6_Ventes;

select 'SQL Server source' as Location,count(*) as Total
from dbo.Ventes2024
UNION ALL
select 'Oracle',cast(CNT as int)
from OPENQUERY(ORA_TP6,'select count(*) CNT from TP6.SALES_EXPORT')
UNION ALL
select 'PostgreSQL',cast(cnt as int)
from OPENQUERY(PG_TP6,'select count(*) cnt from public.sales_export');
```

All three counts are identical:



The Excel destination's **View Existing Data** option displays the three exported sales, with identifiers 1, 2 and 30 and their corresponding values.



The Access destination also completed successfully and reported three rows written in the full Data Flow execution. Together, the execution results, CSV inspection, Excel preview and database counts confirm that the three source sales were exported to all five destinations.

Exercise 2: Importing PostgreSQL sales through FTP

Objective and environment

This exercise consolidates sales from PostgreSQL and SQL Server into one destination table. PostgreSQL data is exported to CSV on Ubuntu, transferred to Windows through FTP and loaded into SQL Server using SSIS. Two additional sales are then inserted locally, with a `Source` column identifying the origin of each row.

The lab uses two virtual machines on the same network:

Machine	Address	Role
Ubuntu 24.04 LTS	`192.168.11.105`	PostgreSQL 16, CSV export and vsftpd server
Windows 11	`192.168.11.104`	SQL Server, Visual Studio and SSIS execution

The Windows VM hosts SQL Server in this implementation.

PostgreSQL source and CSV export

On Ubuntu, the `tp7_sales` database contains the table `public.sales`, with the following fields:

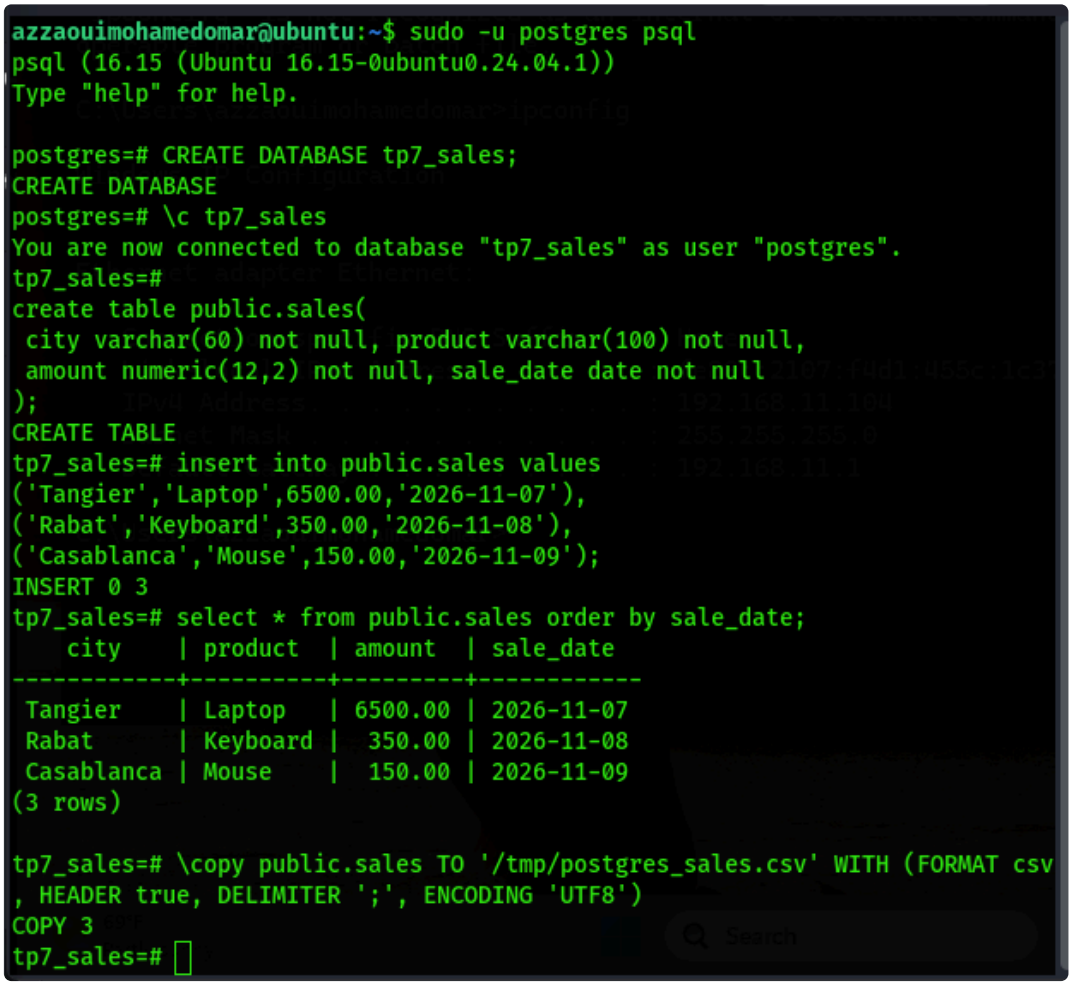
Column	PostgreSQL type
city	`varchar(60)`
product	`varchar(100)`
amount	`numeric(12,2)`
sale_date	`date`

Three sales were inserted and exported from psql using `\copy`:

```
\copy public.sales TO '/tmp/postgres_sales.csv' WITH (FORMAT csv, HEADER
true, DELIMITER ';', ENCODING 'UTF8')
```

The file contains a header and three data rows:

```
city;product;amount;sale_date
Tangier;Laptop;6500.00;2026-11-07
Rabat;Keyboard;350.00;2026-11-08
Casablanca;Mouse;150.00;2026-11-09
```



FTP server and transfer verification

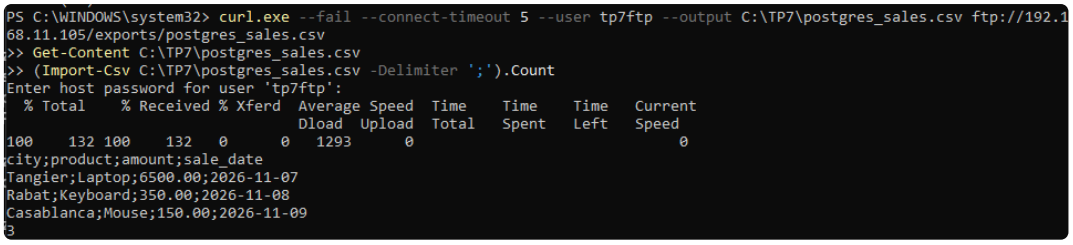
The vsftpd service was configured on Ubuntu for authenticated access using the `tp7ftp` account. Its FTP root is `/srv/ftp`, and the exported file was placed in `/srv/ftp/exports/postgres_sales.csv`. The path visible to FTP clients is therefore `/exports/postgres_sales.csv`.

Passive mode uses ports **30000-30009**, with port **21** handling the control connection. The Ubuntu nftables rules were updated to allow these ports from the Windows address `192.168.11.104` while retaining the existing firewall rules.

Before configuring SSIS, the download was tested from Windows:

```
curl.exe --fail --connect-timeout 5 --user tp7ftp --output
C:\TP7\postgres_sales.csv ftp://192.168.11.105/exports/postgres_sales.csv
Get-Content C:\TP7\postgres_sales.csv
(Import-Csv C:\TP7\postgres_sales.csv -Delimiter ';').Count
```

The transfer succeeded. The downloaded file contains the expected header and sales, and `Import-Csv` returns **3**.



SQL Server destination

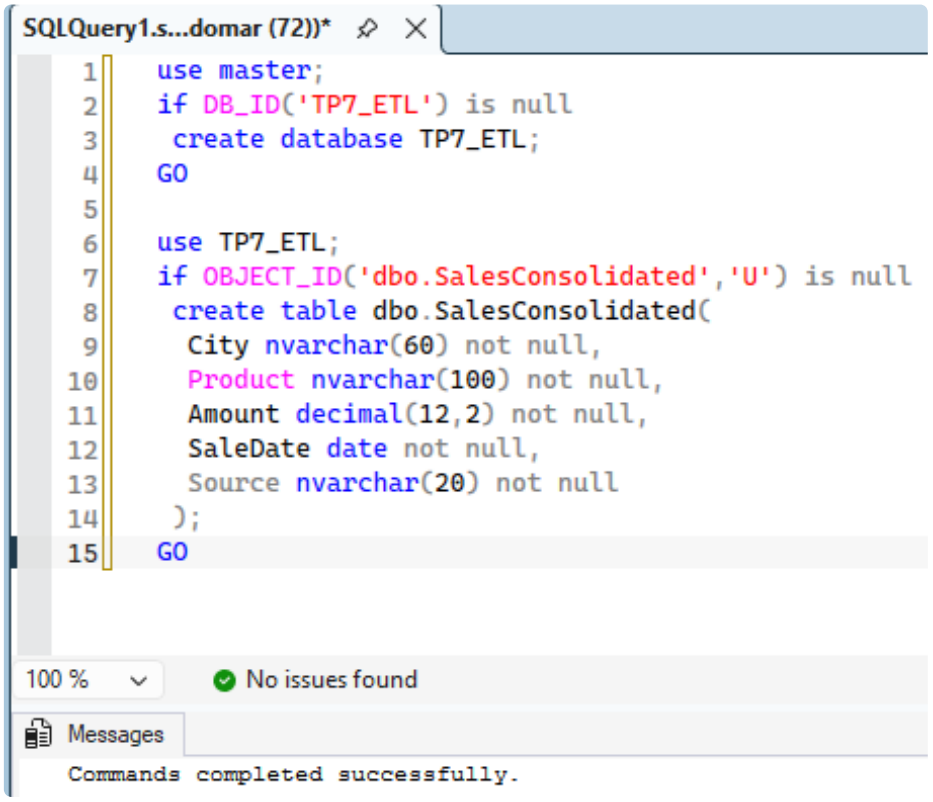
On the default SQL Server instance, the database `TP7_ETL` and destination table `dbo.SalesConsolidated` were created. The table contains the four sales fields and an additional `Source` column:

```
use master;
if DB_ID('TP7_ETL') is null
 create database TP7_ETL;
GO

use TP7_ETL;
if OBJECT_ID('dbo.SalesConsolidated','U') is null
 create table dbo.SalesConsolidated(
  City nvarchar(60) not null,
  Product nvarchar(100) not null,
  Amount decimal(12,2) not null,
  SaleDate date not null,
  Source nvarchar(20) not null
```

```
  );
GO
```

The OLE DB connection uses Windows Authentication and targets `TP7_ETL` on `localhost`. For the local lab server, the connection was configured to trust the server certificate while using mandatory encryption.

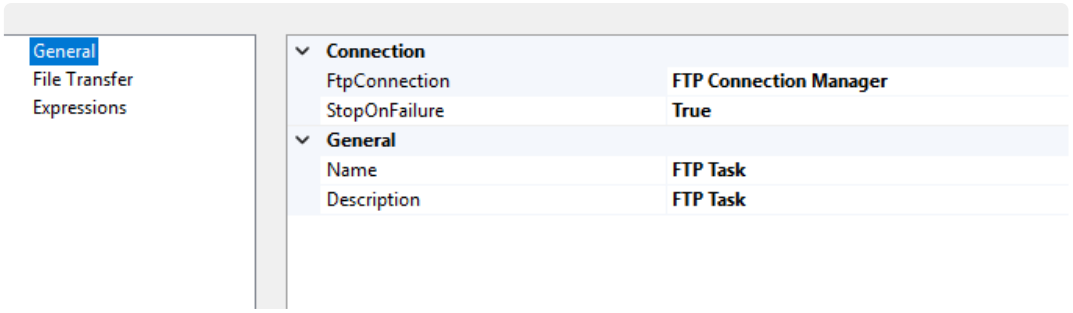


FTP Task configuration

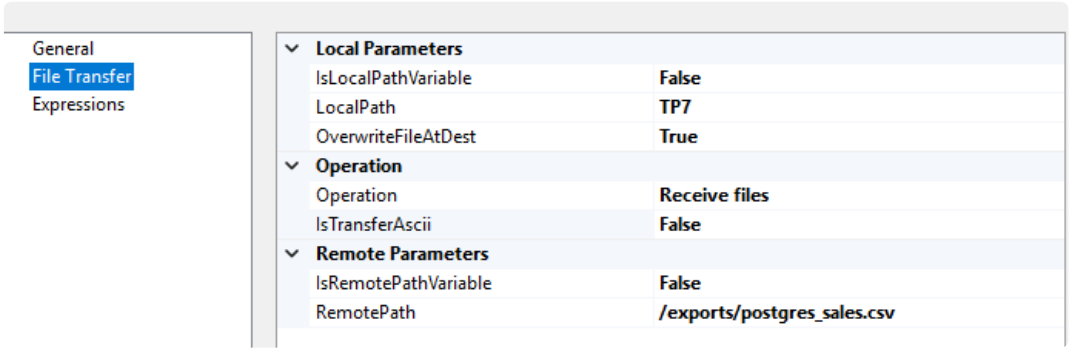
The package `Import_FTP.dtsx` contains three tasks connected by success precedence constraints, in this order: **FTP Task**, **Data Flow Task**, then **Execute SQL Task**.

The FTP Task downloads the CSV before the Data Flow reads it. Its FTP connection manager uses the following settings:

Setting	Value
Server	`192.168.11.105`
Port	`21`
Authentication	FTP account `tp7ftp`
Anonymous access	Disabled
Passive mode	Enabled



The file transfer operation is **Receive files**. The remote path is `/exports/postgres_sales.csv`, and the local destination is `C:\TP7`, referenced through a File connection manager. Overwriting the destination file is enabled, and ASCII transfer is disabled to preserve the file contents.



Data Flow configuration

The Data Flow connects a **Flat File Source** to a **Derived Column** transformation and then an **OLE DB Destination**.

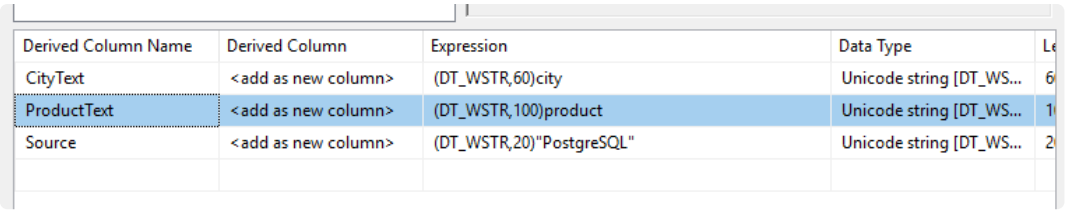
The Flat File Source reads `C:\TP7\postgres_sales.csv` using UTF-8 encoding, semicolon separators and Linux line endings (`LF`). The first row supplies the column names and is excluded from the imported data.

The source metadata defines `city` and `product` as strings, `amount` as a numeric value with precision 12 and scale 2, and `sale_date` as a database date.

The Derived Column transformation creates three new columns:

New column	Expression	Purpose
CityText	`(DT_WSTR,60)city`	Convert city text to Unicode
ProductText	`(DT_WSTR,100)product`	Convert product text to Unicode
Source	`(DT_WSTR,20)"PostgreSQL"`	Identify the origin of imported rows

These conversions match the SQL Server `nvarchar` columns. The original amount and date values pass through unchanged.



Derived Column Name	Derived Column	Expression	Data Type	Le
CityText	<add as new column>	(DT_WSTR,60)city	Unicode string [DT_WS...	6
ProductText	<add as new column>	(DT_WSTR,100)product	Unicode string [DT_WS...	1
Source	<add as new column>	(DT_WSTR,20)"PostgreSQL"	Unicode string [DT_WS...	2

The OLE DB Destination uses **Table or view - fast load** and writes to `[dbo].[SalesConsolidated]` with these mappings:

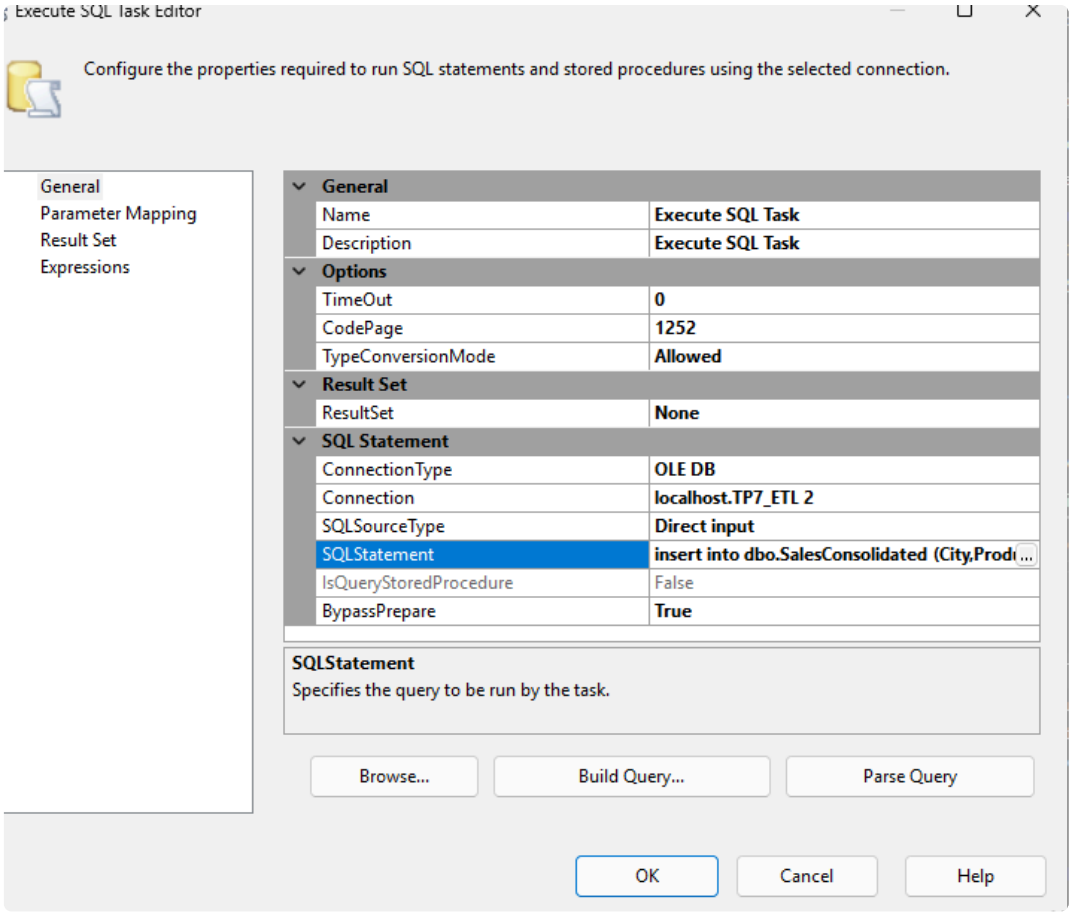
Input column	Destination column
CityText	City
ProductText	Product
amount	Amount
sale_date	SaleDate
Source	Source

Inserting the local SQL Server sales

After the Data Flow succeeds, an **Execute SQL Task** inserts two locally defined sales into the same destination table. It uses the OLE DB connection to `TP7_ETL`, direct SQL input and no result set.

```
insert into dbo.SalesConsolidated
 (City,Product,Amount,SaleDate,Source)
values
 (N'Tangier',N'Monitor',1800.00,'20261109',N'SQLServer'),
 (N'Rabat',N'Headset',450.00,'20261109',N'SQLServer');
```

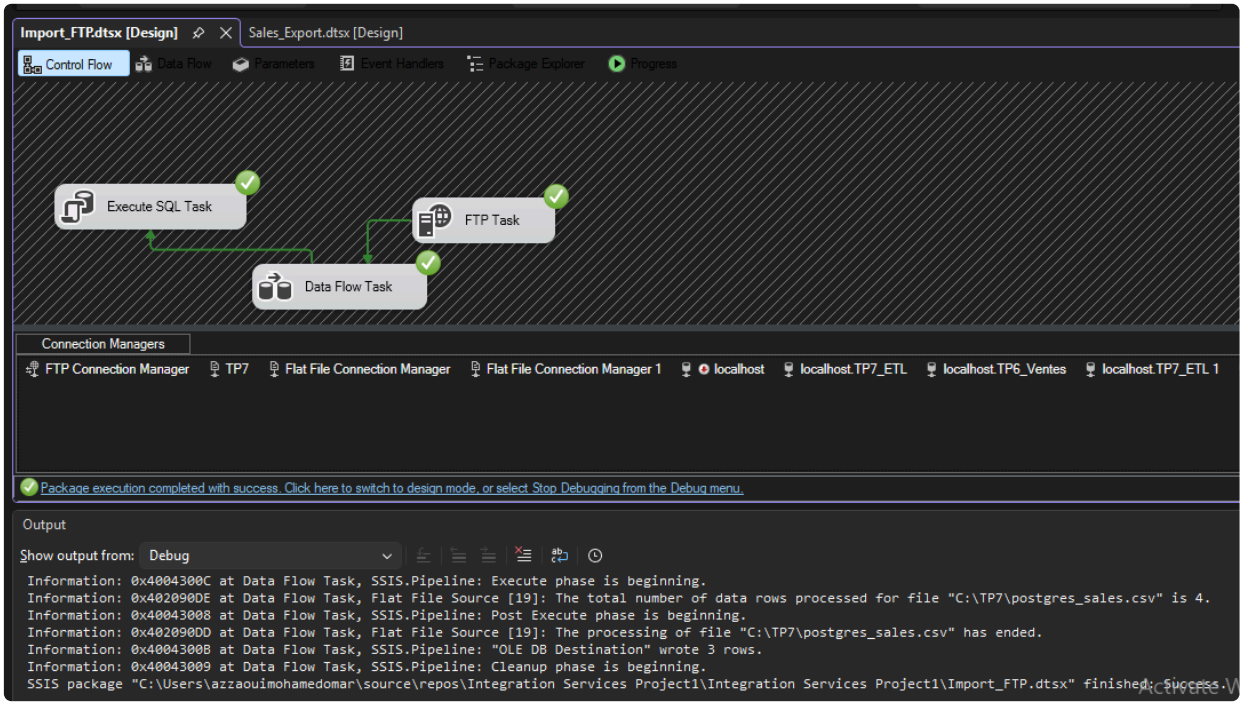
The value `SQLServer` distinguishes these rows from the three imported PostgreSQL sales.



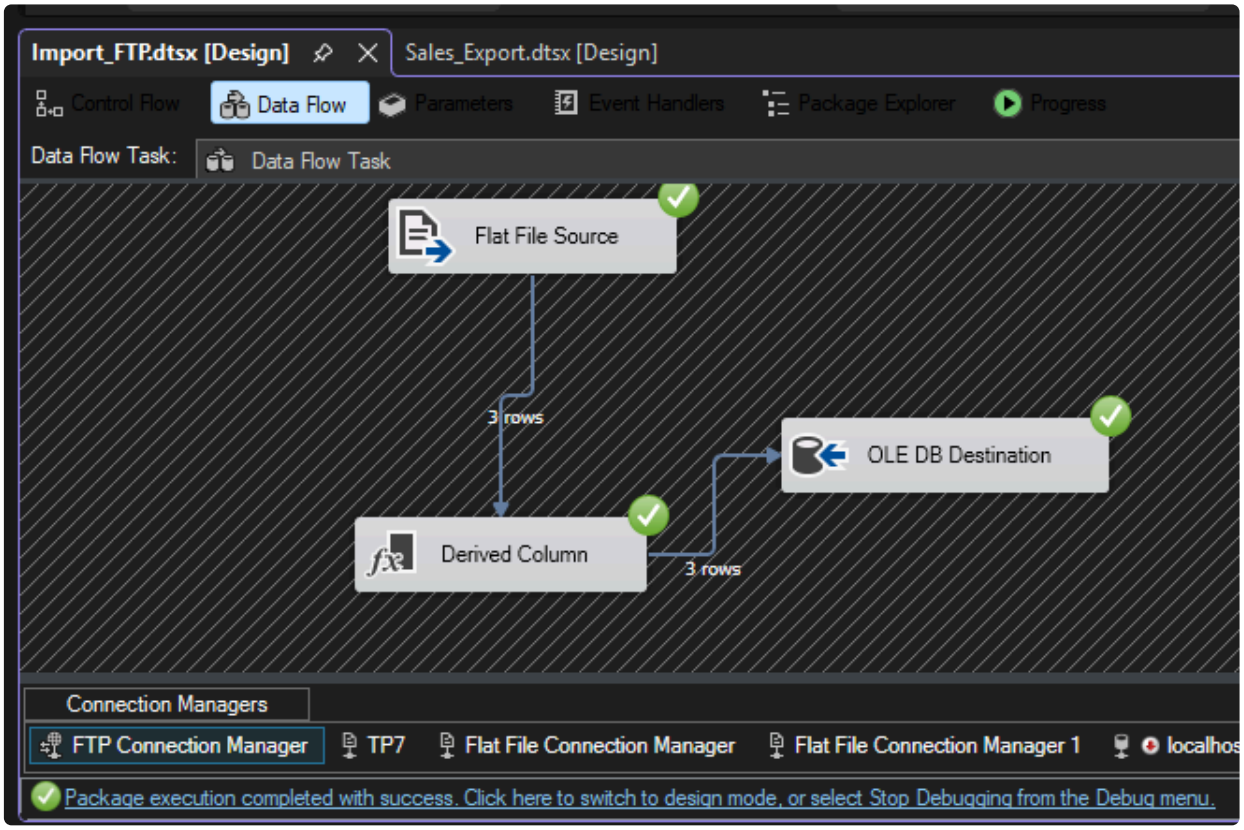
Package execution

Before the final execution, the test destination was cleared to avoid retaining rows from previous runs. The package then completed successfully in the required order: download the CSV, import its three rows and insert the two local sales.

All three Control Flow tasks show successful execution. The output confirms that the OLE DB Destination wrote three rows and that `Import_FTP.dtsx` finished with **Success**.



The Data Flow execution shows three rows passing from the Flat File Source through the Derived Column transformation to the OLE DB Destination.



Verification of the consolidated data

The destination was checked in SSMS using a detailed query followed by a count grouped by source:

```
use TP7_ETL;

select City,Product,Amount,SaleDate,Source
from dbo.SalesConsolidated
order by Source,City;

select Source,count(*) as Total
from dbo.SalesConsolidated
group by Source;
```

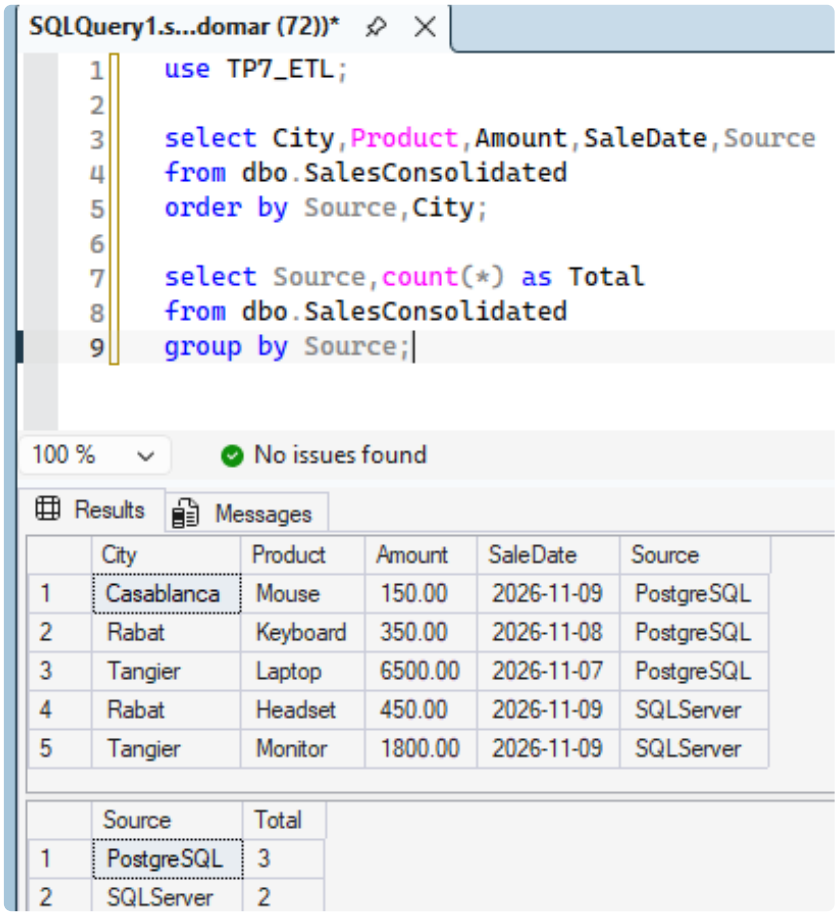
The detailed results contain the expected five sales:

City	Product	Amount	SaleDate	Source
Casablanca	Mouse	150.00	2026-11-09	PostgreSQL
Rabat	Keyboard	350.00	2026-11-08	PostgreSQL
Tangier	Laptop	6500.00	2026-11-07	PostgreSQL
Rabat	Headset	450.00	2026-11-09	SQLServer

City	Product	Amount	SaleDate	Source
Tangier	Monitor	1800.00	2026-11-09	SQLServer

The grouped query confirms their distribution:

Source	Total
PostgreSQL	3
SQLServer	2



The final table combines all five sales while preserving their origin. The FTP Task handles file transfer, the Data Flow parses and loads the PostgreSQL data, and the Execute SQL Task adds the local sales. This package appends rows on execution, so repeating the same import requires clearing the test data or adding duplicate-handling logic.